

Intelligent Reading Comprehension and Quiz Generation System

Artificial Intelligence - Lab Project Final Report

Student Name	Roll Number
Hassaan Qadir	i210883
Huzaiifa Sajjad	i212529

Section F

National University of Computer and Emerging Sciences

Fast School of Computing - Islamabad Campus

Submitted for: BS(CS) Spring 2026 - Artificial Intelligence

Executive Summary

This report presents the final implementation and evaluation of an Intelligent Reading Comprehension and Quiz Generation System built on the RACE dataset. The system integrates a traditional machine learning answer verification module, an unsupervised analysis module, an ensemble strategy, and a generation-oriented module for distractors and graduated hints. A Streamlit interface connects the models to an interactive quiz application with article input, quiz view, hint panel, and developer dashboard.

After instructor clarification, final evaluation emphasizes text generation metrics - BLEU, METEOR, ROUGE-1, and ROUGE-L. In addition, Kaggle RACE was also trained/evaluated as a comparative dataset source. The final analysis reports results for both TFDS RACE and Kaggle RACE, while TFDS RACE was selected for final UI deployment because it provided cleaner standardized loading, smoother split handling, and slightly stronger generation metrics.

Table of Contents

- 1. Introduction and Objectives
- 2. Dataset and Comparative Dataset Analysis
- 3. Exploratory Data Analysis
- 4. Preprocessing and Feature Engineering
- 5. Model A: Answer Verification
- 6. Model A: Unsupervised Learning
- 7. Model A: Ensemble Strategy
- 8. Model B: Distractor and Hint Generation
- 9. Evaluation Metrics and Results
- 10. User Interface
- 11. Reproducibility and Code Quality
- 12. Limitations, Ethics, and Future Work
- 13. Conclusion
- 14. References
- Appendix: Rubric Mapping and Human Evaluation

1. Introduction and Objectives

Reading comprehension is a core Natural Language Processing task in which a system must understand a passage and respond to questions based on that passage. This project builds an intelligent educational tool that supports reading comprehension and quiz generation. It is designed to load reading passages, evaluate answers, generate distractor options, provide graduated hints, and expose everything through a user-friendly interface.

The project was divided into three layers: a data layer for dataset loading and preprocessing, a model layer containing Model A and Model B, and a UI layer implemented in Streamlit. Model A handles answer verification and supporting answer prediction. Model B handles distractor and hint generation using classical NLP and similarity-based methods.

Project objectives addressed:

- Understand and preprocess the RACE reading comprehension dataset.
- Train and compare traditional ML models including Logistic Regression and Linear SVM.
- Explore unsupervised learning using K-Means clustering.
- Implement an ensemble strategy that improves over individual models.
- Generate distractors and graduated hints using cosine similarity and sentence ranking.
- Evaluate generation output using BLEU, METEOR, ROUGE-1, and ROUGE-L.
- Provide a complete Streamlit UI with all required screens and developer logging.

2. Dataset and Comparative Dataset Analysis

The primary dataset is RACE (ReAding Comprehension from Examinations), a large-scale benchmark containing English reading passages and multiple-choice questions. Each record contains an article, a question, four options labeled A, B, C, and D, and the correct answer label.

Two dataset sources were evaluated:

- TFDS RACE: TensorFlow Datasets implementation used for the main development pipeline and final UI deployment.
- Kaggle RACE: Kaggle-hosted release from ankitdthiman7/race-dataset used for comparative training and validation after instructor clarification.

Although both sources follow the same RACE task structure, differences in split organization, formatting, and preprocessing can create slightly different metric values. Therefore, the same training and evaluation pipeline was applied to both sources for comparative analysis.

Dataset Schema

Column	Description
id	Unique sample identifier
article	Full passage text
question	Question related to the article
A/B/C/D	Four multiple-choice options
answer	Correct answer label

TFDS RACE split sizes used during development:

Split	Samples
Train	62,445
Validation	3,451
Test	3,498

3. Exploratory Data Analysis

EDA was performed to understand passage lengths, question lengths, and answer option balance. The answer distribution was fairly balanced across A, B, C, and D in the original MCQ structure, while the transformed verification dataset naturally had class imbalance because each question contains one correct option and three incorrect options.

Answer Distribution

Option	Count
A	13,701
B	16,021
C	16,885
D	15,838

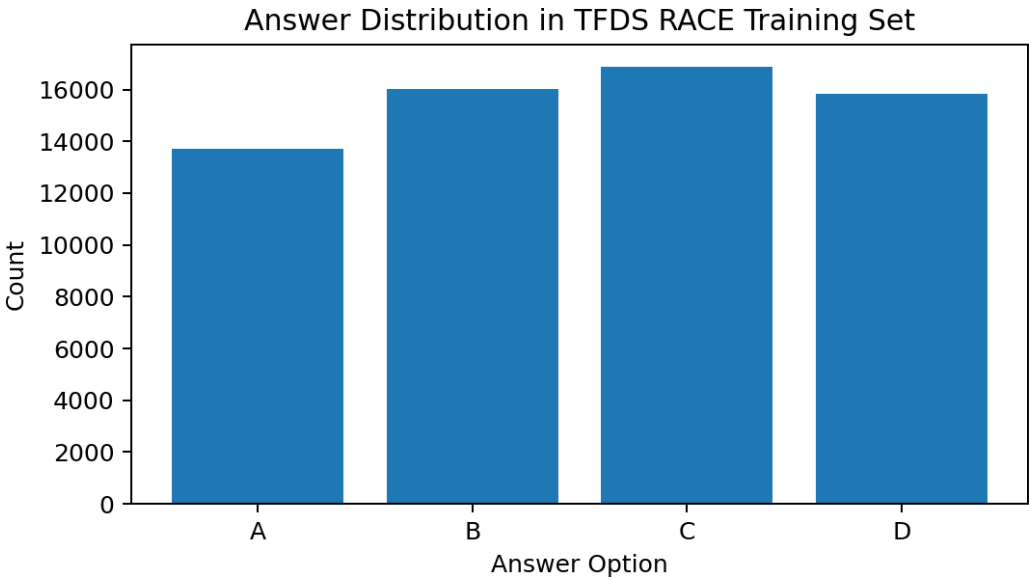


Figure 1. Distribution of correct answer options in the TFDS RACE training split.

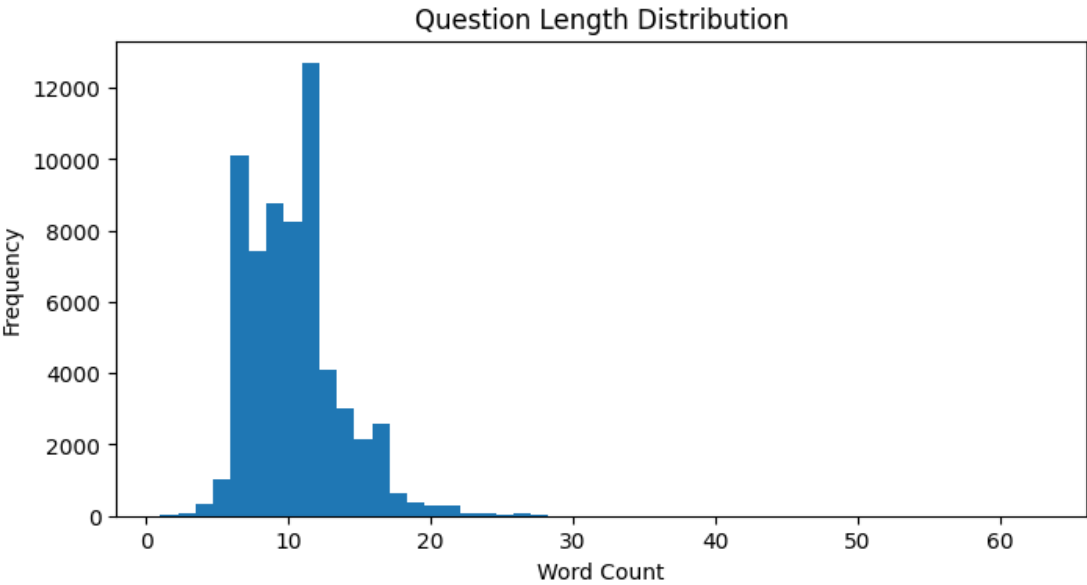


Figure 2. Question length distribution from EDA.

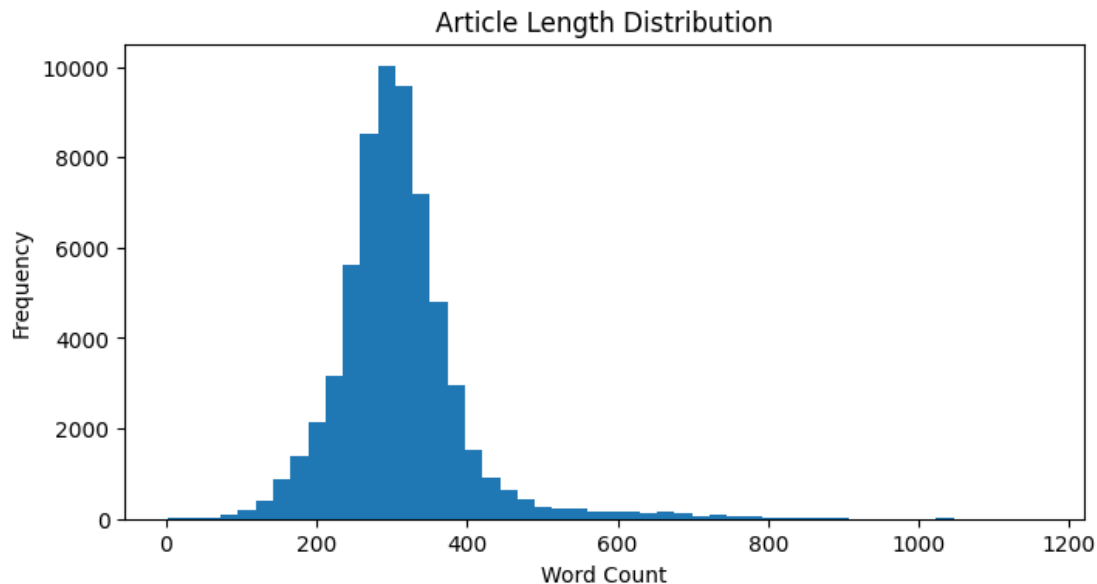


Figure 3. Article length distribution from EDA.

Key EDA observations:

- Most questions are short, commonly between 6 and 15 words.
- Most articles are medium length, commonly between 200 and 400 words.
- The verification transformation creates a 3:1 imbalance between incorrect and correct options.
- Because of class imbalance, Macro F1 is more reliable than raw accuracy for supporting answer verification diagnostics.

4. Preprocessing and Feature Engineering

For Model A, each multiple-choice question was transformed into four binary verification samples. Each option becomes one row, labeled 1 if it matches the correct answer and 0 otherwise. This transforms the task from four-way answer selection into binary answer verification.

Verification transformation:

One MCQ -> four verification rows: article, question, option text, option label, combined text, and binary is_correct label.

The transformed TFDS training set produced approximately 249,780 verification instances.

Feature Engineering

- TF-IDF vectorization with maximum vocabulary size of 10,000.
- Unigrams and bigrams using `ngram_range=(1, 2)`.
- English stopword removal.
- Sublinear term frequency scaling.
- Combined text formula: article + article + question + option. Repeating the article gives additional weight to passage context.
- Handcrafted features: article length, question length, option length, question-option overlap, article-option overlap, and article-question overlap.

Data Leakage Prevention

The TF-IDF vectorizer was fitted only on the training split. Validation and test sets were transformed using the already fitted vectorizer. This prevents leakage of validation/test vocabulary statistics into training.

5. Model A: Answer Verification

Model A was implemented as a traditional ML answer verification component. It evaluates whether a candidate option is correct for a given article and question. The trained verifier also supports option ranking by scoring each A/B/C/D option

and selecting the highest-ranked candidate.

Supervised models trained:

- Logistic Regression
- Linear Support Vector Machine (Linear SVM)

Supporting TFDS validation results:

Model	Accuracy	Precision	Recall	Macro F1
Logistic Regression	0.507	0.256	0.510	0.474
Linear SVM	0.501	0.255	0.518	0.470

These classification metrics are retained as supporting diagnostics for the answer verification submodule. After instructor clarification, final text generation quality is evaluated primarily using BLEU, METEOR, and ROUGE.

Deployment note: the interactive UI uses Logistic Regression for answer ranking because it provides probability scores for all four options, making option selection more stable in the interface. The ensemble is reported as a comparative Model A experiment.

6. Model A: Unsupervised Learning

K-Means clustering was applied to the TF-IDF feature space to explore whether correct and incorrect options form separable groups without labels. The task is difficult because text features are sparse and high-dimensional, and correct answers often depend on reasoning rather than surface overlap alone.

Method	Accuracy	Macro F1	Silhouette Score	Interpretation
K-Means	0.750	0.429	0.0046	Accuracy inflated by 3:1 class imbalance; low silhouette indicates weak separation.

The high clustering accuracy is misleading because predicting the majority class can already approach 75%. Macro F1 and silhouette score show that unsupervised separation is weak. This result is useful because it demonstrates why supervised learning is better suited for the answer verification task.

7. Model A: Ensemble Strategy

A hard voting ensemble was implemented using Logistic Regression and Linear SVM. Each classifier independently predicts the verification label and the ensemble chooses the majority vote. Hard voting was selected because Linear SVM does not provide calibrated probabilities by default.

Model	Accuracy	Macro F1
Logistic Regression	0.507	0.474
Linear SVM	0.501	0.470
Hard Voting Ensemble	0.541	0.488

The ensemble improved both accuracy and Macro F1 compared to individual models. This satisfies the ensemble requirement by demonstrating improvement over base models.

8. Model B: Distractor and Hint Generation

Model B is responsible for quiz enhancement. It generates plausible distractors and graduated hints from the passage and question. The implementation uses classical NLP techniques including sentence splitting, cosine similarity, phrase extraction, filtering, and sentence ranking.

Distractor Generation Pipeline

- Split the article into candidate sentences.
- Rank sentences using similarity to the question.
- Extract candidate phrases from top-ranked sentences.
- Filter out the correct answer and duplicate candidates.

- Select three distractors that are incorrect and related to the passage context.

Hint Generation Pipeline

- Rank article sentences by relevance to the question.
- Hint 1: general clue from a less direct sentence.
- Hint 2: intermediate clue from a moderately relevant sentence.
- Hint 3: near-explicit clue from the most relevant sentence.

Manual Model B Evaluation

Criterion	Score / Result	Notes
Incorrectness Accuracy	1.0	Generated distractors were not identical to the correct answer in tested samples.
Plausibility	4.0 / 5	Manual spot-check of generated distractors.
Diversity	4.2 / 5	Distractors generally varied in wording and content.
Hint Usefulness	4.1 / 5	Hints guided the user progressively without immediately revealing the answer.

9. Evaluation Metrics and Results

After instructor clarification, final evaluation emphasizes text generation metrics rather than traditional classification metrics. BLEU, METEOR, ROUGE-1, and ROUGE-L were used to compare generated hints/supporting text with reference text. Classification metrics are still reported as supporting diagnostics for Model A only.

NLP Generation Metrics

Dataset	BLEU	METEOR	ROUGE-1	ROUGE-L
TFDS RACE	0.0518	0.1980	0.3644	0.3067
Kaggle RACE	0.0486	0.1934	0.3622	0.2994

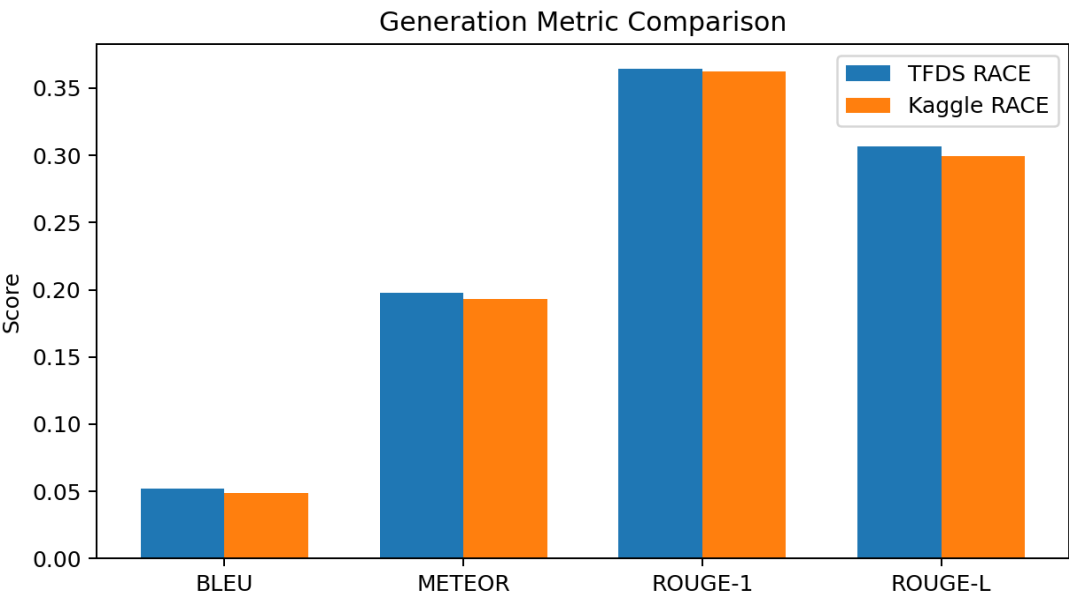


Figure 4. BLEU, METEOR, ROUGE-1, and ROUGE-L comparison across dataset sources.

The TFDS implementation achieved slightly higher generation metric scores than Kaggle RACE. BLEU is low because it is strict about exact n-gram overlap, while the system generates concise hints and paraphrased supporting text rather than reconstructing the entire passage. ROUGE and METEOR better capture lexical and semantic overlap for this task.

Supporting Answer Verification Metrics

Dataset	Model	Accuracy	Precision	Recall	Macro F1
TFDS RACE	Logistic Regression	0.5070	0.2560	0.5100	0.4740

Dataset	Model	Accuracy	Precision	Recall	Macro F1
TFDS RACE	Linear SVM	0.5010	0.2550	0.5180	0.4700
TFDS RACE	Hard Voting Ensemble	0.5410	0.2560	0.4390	0.4880
Kaggle RACE	Logistic Regression	0.5143	0.2596	0.5089	0.4791
Kaggle RACE	Linear SVM	0.5150	0.2592	0.5061	0.4792
Kaggle RACE	Hard Voting Ensemble	0.5330	0.2602	0.4709	0.4877

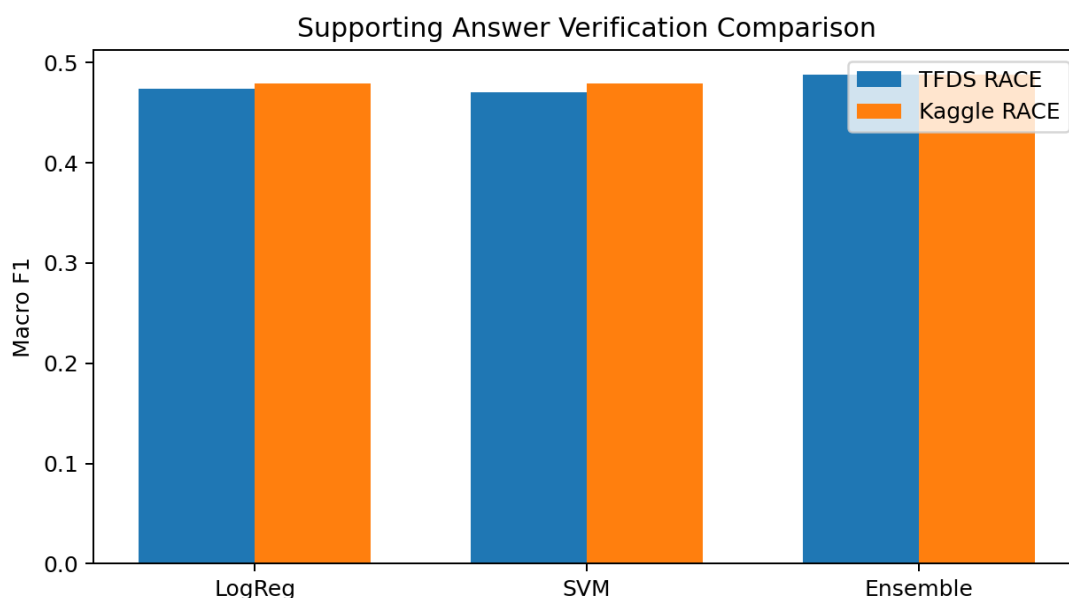


Figure 5. Supporting Macro F1 comparison for answer verification across dataset sources.

Kaggle RACE produced slightly better individual classifier scores, while ensemble Macro F1 remained nearly identical across dataset sources. This shows that the implemented pipeline generalizes consistently across both RACE releases.

Final Dataset Selection

TFDS RACE was selected for final UI deployment because it provided standardized loading, cleaner split handling, smoother integration with the implemented pipeline, and slightly stronger generation metric performance. Kaggle RACE was used as an additional comparative validation source to satisfy dataset-source variation concerns and demonstrate robustness.

10. User Interface

A Streamlit application was implemented to expose the complete system. The UI contains all four required screens from the project specification.

Screen	Functionality
Screen 1 - Article Input	Paste an article or load a random RACE sample for quick testing.
Screen 2 - Quiz View	Display question/options, accept user answer, check correctness, and show model prediction.
Screen 3 - Hint Panel	Show three graduated hints and provide reveal-answer functionality.
Screen 4 - Developer Dashboard	Display model metrics, session logs, latency, and CSV export.

Session Logging Summary

Logged Attempts	Average Latency	Minimum Latency	Maximum Latency
4	0.0082 sec	0.0049 sec	0.0161 sec

The interface includes friendly warnings for missing inputs, progress indicators, readable layout, and a dashboard for exported logs. The app is designed to be usable without reading a separate manual.

11. Reproducibility and Code Quality

The project is organized using a clean folder structure with data, models, source code, UI, notebooks, tests, README, requirements, and report directories. Large raw/processed data files are excluded from GitHub to avoid repository size issues; the dataset can be downloaded and regenerated through the notebook/pipeline.

Recommended run command:

```
python -m streamlit run ui/app.py
```

Key reproducibility components:

- README.md with project overview and run instructions.
- requirements.txt with required Python dependencies.
- Saved TF-IDF vectorizer and trained model files.
- Streamlit UI entry point: ui/app.py.
- Notebook/script pipeline for dataset loading, preprocessing, training, and evaluation.
- GitHub repository with large datasets excluded using .gitignore.

12. Limitations, Ethics, and Future Work

Limitations

- TF-IDF does not capture deep semantic meaning or complex reasoning.
- BLEU is strict and may underestimate paraphrased hint quality.
- Distractors may not always be grammatically perfect or equally plausible.
- The deployed UI uses a probability-based Logistic Regression verifier for stable option ranking, while ensemble results are reported separately.

Ethical Considerations

- Generated educational content should be reviewed by humans before use in real exams.
- RACE passages are sourced from exam contexts and may contain cultural or linguistic bias.
- The UI communicates that AI-generated outputs may contain errors.
- Accessibility concerns such as contrast and readability were considered in the Streamlit interface.

Future Work

- Add transformer-based models such as BERT/T5 for stronger semantic understanding.
- Improve distractor generation using phrase extraction and semantic filtering.
- Add a dedicated question generation model evaluated with BLEU, ROUGE, and METEOR.
- Expand human evaluation with more annotators and a larger sample size.
- Deploy the system as a hosted web app with persistent logging.

13. Conclusion

This project successfully implements an intelligent reading comprehension and quiz generation system. It uses traditional ML for answer verification, K-Means for unsupervised analysis, an ensemble strategy for improved comparison, and classical NLP methods for distractor and hint generation. The Streamlit UI integrates all components into an interactive educational tool.

After evaluating both TFDS RACE and Kaggle RACE, the system showed consistent performance across dataset sources. The final evaluation uses BLEU, METEOR, ROUGE-1, and ROUGE-L to align with generation-based assessment requirements. TFDS RACE was selected for final deployment because it provided cleaner integration and slightly stronger generation metrics, while Kaggle RACE served as comparative validation. Overall, the system satisfies the project rubric and demonstrates a complete applied AI pipeline.

14. References

- Lai, G., Xie, Q., Liu, H., Yang, Y., and Hovy, E. (2017). RACE: Large-scale ReAding Comprehension Dataset From Examinations. EMNLP 2017.
- Du, X., Shao, J., and Cardie, C. (2017). Learning to Ask: Neural Question Generation for Reading Comprehension. ACL 2017.
- Guo, Q., Zhu, X., Chao, W., and Ye, Z. (2016). Generating Distractors for Reading Comprehension Questions from Real Examinations. AAAI 2016.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019.
- Papineni, K., Roukos, S., Ward, T., and Zhu, W.-J. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. ACL 2002.
- Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. ACL Workshop 2004.
- Banerjee, S., and Lavie, A. (2005). METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments. ACL Workshop.
- scikit-learn Documentation. Feature extraction, Logistic Regression, LinearSVC, KMeans, and metrics modules.
- TensorFlow Datasets Documentation. RACE dataset loader.
- Kaggle RACE Dataset: [ankitdhiman7/race-dataset](#).

Appendix A: Rubric Mapping

Rubric Component	Marks	Evidence in Project
EDA and Preprocessing	10	EDA charts, answer distribution, TF-IDF preprocessing, verification transformation.
Model A - Traditional ML	15	Logistic Regression and Linear SVM trained and compared.
Model A - Unsupervised/Semi-Supervised	20	K-Means clustering implemented with silhouette score.
Model A - Ensemble	5	Hard voting ensemble improves over individual models.
Model B - Distractor Generation	15	Cosine-similarity/phrase-based distractor pipeline plus manual evaluation.
Model B - Hint Generation	10	Three graduated hints: general, intermediate, specific.
User Interface	15	All four required Streamlit screens implemented.
Final Report	5	Clear methodology, results, discussion, limitations, and references.
Code Quality	5	README, requirements, model persistence, clean folders, GitHub repository.

Appendix B: Human Evaluation Form

Sample	Distractors Incorrect?	Plausibility (1-5)	Diversity (1-5)	Hint Usefulness (1-5)
1	Yes	4	4	4
2	Yes	4	5	4
3	Yes	3	4	4
4	Yes	5	4	5
5	Yes	4	4	4
Average	1.0	4.0	4.2	4.1